

<h1 id="Global-Misinformation-Analysis:-GDELT-Based-Event-Dataset">Global Misinformation Analysis: GDELT-Based Event Dataset</h1> <p>**Research questions**</p>

1. What types of misinformation themes appear in global news reporting?
2. Where are these events geographically concentrated?
3. Under what political, economic, or public health conditions does misinformation activity increase?

<p>**Methods:** LDA topic modelling (theme identification) · DBSCAN / Moran's I / KDE (spatial hotspots) · Poisson / Negative Binomial regression (contextual drivers)</p> <p>**Data pipeline:** this notebook is Stage 3 of a three-stage pipeline:</p>

1. `Extraction.py` — downloads real records directly from GDELT's live GKG v2.1 archive, filtered on documented real theme codes
2. `Clean_dataset.py` — parses GDELT's own `V2Themes` / `V2Locations` / `V2Tone` fields into the analytical dataset
3. **This notebook** — runs the full analysis on the output of step 2

<p>**Input file:** `gdelt_cleaned_dataset.csv` (the output of `Clean_dataset.py` -- run steps 1 and 2 first if you don't have this file yet)</p>

<h2 id="1.-Environment-setup">1. Environment setup</h2> <p>Run the cell below **once** in an Anaconda Prompt / terminal (with your environment activated) if any of these packages are missing. It is commented out so it doesn't run automatically inside the notebook.</p>

```
In [1]: # Run this cell once to install missing packages directly into THIS notebook's kernel.
# After it finishes, go to Kernel -> Restart Kernel, then run the notebook from the top.

%pip install pandas numpy matplotlib seaborn scikit-learn statsmodels gensim scipy
%pip install geopandas libpysal esda shapely
```

Requirement already satisfied: pandas in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (2.3.3)
Requirement already satisfied: numpy in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (2.2.6)
Requirement already satisfied: matplotlib in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (3.10.9)
Requirement already satisfied: seaborn in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (0.13.2)
Requirement already satisfied: scikit-learn in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (1.7.2)
Requirement already satisfied: statsmodels in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (0.14.6)
Requirement already satisfied: gensim in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (4.4.0)
Requirement already satisfied: scipy in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (1.15.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas) (2026.3)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (1.3.2)
Requirement already satisfied:ycler>=0.10 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow=8 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: joblib=1.2.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: patsy>=0.5.6 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from statsmodels) (1.0.2)
Requirement already satisfied: smart_open>=1.8.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from gensim) (8.0.1)
Requirement already satisfied: six>=1.5 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: wrapt in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from smart_open>=1.8.1->gensim) (2.2.2)
Note: you may need to restart the kernel to use updated packages.
Requirement already satisfied: geopandas in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (1.1.4)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: libpysal in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (4.13.0)
Requirement already satisfied: esda in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (2.7.0)
Requirement already satisfied: shapely in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (2.1.2)
Requirement already satisfied: numpy>=1.24 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from geopandas) (2.2.6)
Requirement already satisfied: packaging in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from geopandas) (26.2)
Requirement already satisfied: pandas>=2.0.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from geopandas) (2.3.3)
Requirement already satisfied: beautifulsoup4>=4.10 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from libpysal) (4.15.0)
Requirement already satisfied: platformdirs>=2.0.2 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from libpysal) (4.10.0)
Requirement already satisfied: requests>=2.27 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from libpysal) (2.34.2)
Requirement already satisfied: scipy>=1.8 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from libpysal) (1.15.2)
Requirement already satisfied: scikit-learn>=1.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from libpysal) (1.7.2)
Requirement already satisfied: soupsieve>=1.6.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from beautifulsoup4>=4.10->libpysal) (2.8.4)
Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from beautifulsoup4>=4.10->libpysal) (4.16.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas>=2.0.0->geopandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas>=2.0.0->geopandas) (2026.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from pandas>=2.0.0->geopandas) (2026.3)
Requirement already satisfied: six>=1.5 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from python-dateutil>=2.8.2->pandas>=2.0.0->geopandas) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from requests>=2.27->libpysal) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from requests>=2.27->libpysal) (3.18)
Requirement already satisfied: urllib3<3,>=1.26 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from requests>=2.27->libpysal) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from requests>=2.27->libpysal) (2026.7.22)
Requirement already satisfied: joblib>=1.2.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from scikit-learn>=1.1->libpysal) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\ezech\anaconda3\envs\msba\lib\site-packages (from scikit-learn>=1.1->libpysal) (3.6.0)

```
In [3]: # Core data handling
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np

# Visualisation
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

# Text / NLP
from gensim import corpora, models
from gensim.models.coherencemodel import CoherenceModel
```

```
# Spatial analysis
import geopandas as gpd
from shapely.geometry import Point
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
import libpysal
from libpysal.weights import KNN
from esda.moran import Moran
from scipy.stats import gaussian_kde

# Statistical modelling
import statsmodels.api as sm
import statsmodels.formula.api as smf

print("All libraries imported successfully.")
```

All libraries imported successfully.

<h2 id="2.-Load-and-inspect-the-analytical-dataset">2. Load and inspect the analytical dataset</h2>

```
In [4]: DATA_PATH = "gdelit_cleaned_dataset.csv" # <-- output of Clean_dataset.py; update path if needed

df_raw = pd.read_csv(DATA_PATH)

print("Shape:", df_raw.shape)
df_raw.head()
```

Shape: (82974, 19)

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>
<thead>
<th></th>
<th>Record_ID</th>
<th>Date</th>
<th>Year</th>
<th>Month</th>
<th>Year_Month</th>
<th>Source</th>
<th>Source_URL</th>
<th>Country</th>
<th>Location_Type</th>
<th>Latitude</th>
<th>Longitude</th>
<th>Themes</th>
<th>Theme_Count</th>
<th>Tone_Overall</th>
<th>Tone_Positive</th>
<th>Tone_Negative</th>
<th>Tone_Polarity</th>
<th>Tone_Activity</th>
<th>Tone_SelfGroup</th>
</thead>
<tbody>
<th>0</th>
<th>1</th>
<th>2</th>
<th>3</th>
<th>4</th>
</tbody>
<tbody>
</tbody>
```

20220101121500-2	2022-01-01 12:15:00	2022	1	2022-01	smdailyjournal.com	https://www.smdailyjournal.com/news/world/pope...	Vatican City	1	41.9000	12.450000	TAX_FNCACT_CARDINAL;WB_1467_EDUCATION_FOR_ALL;...	31	-0.245098	3.676471	3.921569	7.598039	21.323529	0.490196
20220101121500-4	2022-01-01 12:15:00	2022	1	2022-01	townhall.com	https://townhall.com/tipsheet/mattvespa/2022/0...	United States	2	32.9866	-83.648700	HEALTH_VACCINATION;UNGP_HEALTHCARE;TAX_FNCACT_...	35	-2.803738	1.246106	4.049844	5.295950	25.233645	2.024922
20220101121500-5	2022-01-01 12:15:00	2022	1	2022-01	gov.ms	https://www.gov.ms/2022/01/01/new-years-messag...	Montserrat	1	16.7500	-62.200000	SHORTAGE;UNGP_FORESTS_RIVERS_OCEANS;WB_528_SOL...	87	2.583026	4.691618	2.108593	6.800211	22.298366	2.899315
20220101121500-6	2022-01-01 12:15:00	2022	1	2022-01	majorcadailybulletin.com	https://www.majorcadailybulletin.com/news/loca...	Spain	4	38.3462	-0.487722	GENERAL_GOVERNMENT;EPU_POLICY_GOVERNMENT;GENER...	25	3.773585	4.716981	0.943396	5.660377	17.358491	0.000000
20220101121500-8	2022-01-01 12:15:00	2022	1	2022-01	houstonchronicle.com	https://www.houstonchronicle.com/news/article/...	United States	3	44.0805	-103.231000	RURAL;TAX_FNCACT_ASSISTANT;TAX_FNCACT_GUIDE;SO...	53	1.091520	1.595298	0.503778	2.099076	22.670025	2.099076

```
In [5]: df_raw.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 82974 entries, 0 to 82973
Data columns (total 19 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Record_ID       82974 non-null  object
1   Date            82974 non-null  object
2   Year            82974 non-null  int64
3   Month           82974 non-null  int64
4   Year_Month      82974 non-null  object
5   Source          82971 non-null  object
6   Source_URL      82974 non-null  object
7   Country         82974 non-null  object
8   Location_Type   82974 non-null  int64
9   Latitude        82974 non-null  float64
10  Longitude       82974 non-null  float64
11  Themes          82974 non-null  object
12  Theme_Count     82974 non-null  int64
13  Tone_Overall    82974 non-null  float64
14  Tone_Positive   82974 non-null  float64
15  Tone_Negative   82974 non-null  float64
16  Tone_Polarity   82974 non-null  float64
17  Tone_Activity   82974 non-null  float64
18  Tone_SelfGroup  82974 non-null  float64
dtypes: float64(8), int64(4), object(7)
memory usage: 12.0+ MB
```

```
In [6]: df_raw.describe(include="all").T
```

Out[6]:

<style>scoped</style> dataframe tbody tr th:only-of-type { vertical-align: middle; }.dataframe thead th { text-align: right; } </thead><thead><th></th><th>count</th><th>unique</th><th>top</th><th>freq</th><th>mean</th><th>std</th><th>min</th><th>25%</th><th>50%</th><th>75%</th><th>max</th></thead><tbody><tr><th>Record_ID</th><th>Date</th><th>Year</th><th>Month</th><th>Year_Month</th><th>Source</th><th>Source_URL</th><th>Country</th><th>Location_Type</th><th>Latitude</th><th>Longitude</th><th>Themes</th><th>Theme_Count</th><th>Tone_Overall</th><th>Tone_Positive</th><th>Tone_Negative</th><th>Tone_Polarity</th><th>Tone_Activity</th><th>Tone_SelfGroup</th></tbody></tbody></table><table><tr><td>82974</td><td>82974</td><td></td><td>20240101121500-863</td><td>1</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974</td><td>728</td><td></td><td>2022-03-17 12:15:00</td><td>271</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>2022.444163</td><td>0.498063</td><td>2022.0</td><td>2022.0</td><td>2022.0</td><td>2023.0</td><td>2024.0</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>6.024875</td><td>3.334654</td><td>1.0</td><td>3.0</td><td>6.0</td><td>9.0</td><td>12.0</td></tr><tr><td>82974</td><td>25</td><td></td><td>2022-01</td><td>5445</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82971</td><td>6696</td><td></td><td>investegate.co.uk</td><td>1497</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974</td><td>82972</td><td></td><td>https://bioprepwatch.com/what-reaction-strateg...</td><td>2</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974</td><td>564</td><td></td><td>United States</td><td>20659</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>2.092186</td><td>1.295624</td><td>1.0</td><td>1.0</td><td>1.0</td><td>3.0</td><td>5.0</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>36.656485</td><td>19.70064</td><td>-85.6221</td><td>30.5833</td><td>39.828175</td><td>51.7333</td><td>81.8333</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>-0.210927</td><td>80.053196</td><td>-175.167</td><td>-79.4167</td><td>-2.98333</td><td>77.0</td><td>179.383</td></tr><tr><td>82974</td><td>72268</td><td></td><td>EPU_POLICY_REGULATORY;TAX_FNCACT_DIRECTOR;WB_6...</td><td>351</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td><td>NaN</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>47.229277</td><td>29.551283</td><td>1.0</td><td>27.0</td><td>41.0</td><td>61.0</td><td>639.0</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>-1.389047</td><td>3.283277</td><td>-25.454545</td><td>-3.521779</td><td>-1.32848</td><td>0.768351</td><td>20.0</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>2.762502</td><td>1.579263</td><td>0.0</td><td>1.666667</td><td>2.469136</td><td>3.521127</td><td>20.0</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>4.151549</td><td>2.456731</td><td>0.0</td><td>2.244389</td><td>3.855572</td><td>5.681818</td><td>25.454545</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>6.914051</td><td>2.505851</td><td>0.0</td><td>5.204461</td><td>6.743979</td><td>8.450704</td><td>26.086957</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>21.526344</td><td>3.289878</td><td>0.0</td><td>19.518149</td><td>21.646746</td><td>23.636364</td><td>39.423077</td></tr><tr><td>82974.0</td><td>NaN</td><td></td><td>NaN</td><td>NaN</td><td>0.859846</td><td>0.991705</td><td>0.0</td><td>0.125156</td><td>0.561369</td><td>1.232666</td><td>9.712076</td></tr></table>

In [7]:

```
# Confirm the key columns we rely on throughout the notebook are present and typed correctly
required_cols = [
    "Record_ID", "Date", "Year", "Month", "Year_Month",
    "Country", "Location_Type", "Latitude", "Longitude",
    "Themes", "Theme_Count",
    "Tone_Overall", "Tone_Positive", "Tone_Negative",
    "Tone_Polarity", "Tone_Activity", "Tone_SelfGroup",
    "Source", "Source_URL",
]
missing_cols = [c for c in required_cols if c not in df_raw.columns]
print("Missing required columns:", missing_cols if missing_cols else "None")

df_raw.dtypes[required_cols]
```

Missing required columns: None

Out[7]:

Record_ID	object
Date	object
Year	int64
Month	int64
Year_Month	object
Country	object
Location_Type	int64
Latitude	float64
Longitude	float64
Themes	object
Theme_Count	int64
Tone_Overall	float64
Tone_Positive	float64
Tone_Negative	float64
Tone_Polarity	float64
Tone_Activity	float64
Tone_SelfGroup	float64
Source	object
Source_URL	object
dtype:	object

3. Clean and preprocess the data

In [8]:

```
df = df_raw.copy()

# --- Deduplicate on the unique event identifier ---
before = len(df)
df = df.drop_duplicates(subset=["Record_ID"])
print(f'Removed {before - len(df)} duplicate rows (by Record_ID). Remaining: {len(df)}')

# NOTE: unlike the old synthetic dataset, there is no Misinfo_Flag column to filter on here.
# Every row in gdelt_cleaned_dataset.csv already matched at least one documented
# misinformation-related GKG theme code during extraction (see Extraction.py's
# MISINFO_THEMES list), so no further filtering is needed at this stage.
```

Removed 0 duplicate rows (by Record_ID). Remaining: 82974

In [9]:

```
# --- Missing values ---
na_summary = df.isna().sum()
```



```
na_summary = na_summary[na_summary > 0]
print("Columns with missing values:")
print(na_summary if len(na_summary) else "None - dataset is complete")

# Drop rows that are unusable for spatial analysis (no coordinates) or have no theme info
df = df.dropna(subset=["Latitude", "Longitude", "Themes"])

# For any missing tone scores, fill with the column median rather than dropping the event
tone_cols = ["Tone_Overall", "Tone_Positive", "Tone_Negative", "Tone_Polarity", "Tone_Activity", "Tone_SelfGroup"]
for c in tone_cols:
    if df[c].isna().any():
        df[c] = df[c].fillna(df[c].median())
```

Columns with missing values:

Source 3
dtype: int64

```
In [10]: # --- Standardise country names (extend this mapping as new spellings appear in your data) ---
country_map = {
    "United States": "USA",
    "United States of America": "USA",
    "U.S.": "USA",
    "UK": "United Kingdom",
    "Great Britain": "United Kingdom",
}
df["Country"] = df["Country"].replace(country_map)
print("Unique countries after standardisation:", df["Country"].nunique())
```

Unique countries after standardisation: 563

```
In [11]: # --- Time variables ---
df["Date"] = pd.to_datetime(df["Date"])

# Rebuild Year/Month/Year_Month from Date to guarantee internal consistency
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Year_Month"] = df["Date"].dt.to_period("M").astype(str)

print(df[["Date", "Year", "Month", "Year_Month"]].head())
print("Date range:", df["Date"].min().date(), "to", df["Date"].max().date())
```

	Date	Year	Month	Year_Month
0	2022-01-01 12:15:00	2022	1	2022-01
1	2022-01-01 12:15:00	2022	1	2022-01
2	2022-01-01 12:15:00	2022	1	2022-01
3	2022-01-01 12:15:00	2022	1	2022-01
4	2022-01-01 12:15:00	2022	1	2022-01

Date range: 2022-01-01 to 2024-01-01

4. Theme identification via LDA (misinformation types)

Each event's Themes field (e.g. "ELECTION; CYBER_ATTACK; VACCINE_MISINFORMATION") is treated as a short "document" made up of GDELT theme tags. LDA is fit on this theme-token corpus to discover latent misinformation topics, and each event is assigned its single most probable topic.

```
In [12]: # --- Tokenise the Themes field into lists ---
df["Theme_Tokens"] = df["Themes"].apply(
    lambda s: [t.strip() for t in str(s).split(";") if t.strip()]
)

print(df[["Themes", "Theme_Tokens"]].head(3))
```

	Themes	\
0	TAX_FNCACT_CARDINAL;WB_1467_EDUCATION_FOR_ALL;...	
1	HEALTH_VACCINATION;UNGP_HEALTHCARE;TAX_FNCACT_...	
2	SHORTAGE;UNGP_FORESTS_RIVERS_OCEANS;WB_528_SOL...	

	Theme_Tokens
0	[TAX_FNCACT_CARDINAL, WB_1467_EDUCATION_FOR_AL...
1	[HEALTH_VACCINATION, UNGP_HEALTHCARE, TAX_FNCA...
2	[SHORTAGE, UNGP_FORESTS_RIVERS_OCEANS, WB_528_...

```
In [13]: # --- Build gensim dictionary and bag-of-words corpus ---
texts = df["Theme_Tokens"].tolist()

dictionary = corpora.Dictionary(texts)
corpus = [dictionary.doc2bow(text) for text in texts]

print(f"Vocabulary size: {len(dictionary)}")
print(f"Number of documents (events): {len(corpus)}")
```

Vocabulary size: 10733

Number of documents (events): 82974

```
In [14]: # --- Try a range of topic counts and compare coherence (c_v) ---
topic_range = range(3, 9)
coherence_scores = []

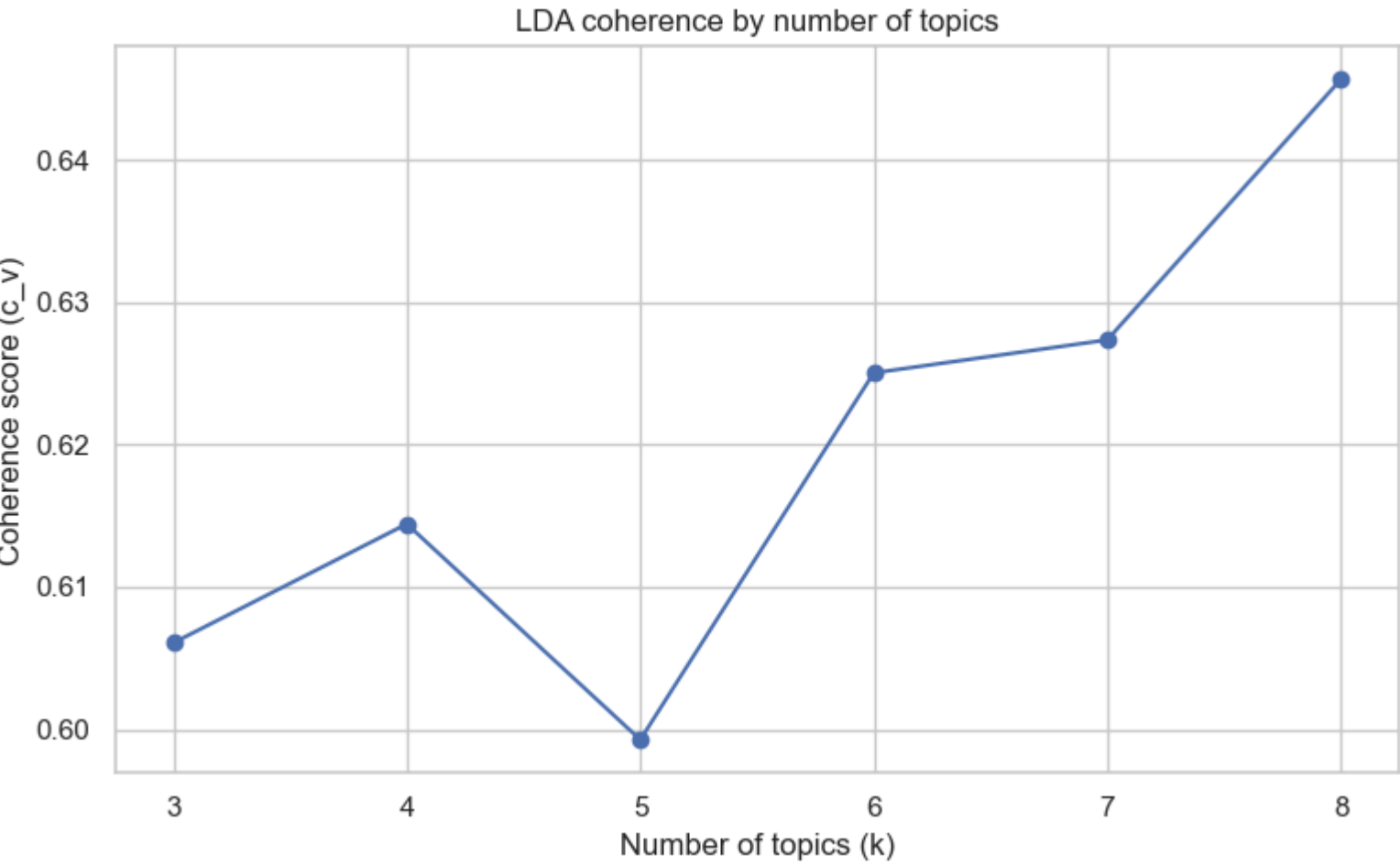
for k in topic_range:
    lda_k = models.LdaModel(
        corpus=corpus,
        id2word=dictionary,
        num_topics=k,
        random_state=42,
        passes=20,
        alpha="auto",
        per_word_topics=False,
    )
```



```
cm = CoherenceModel(model=lda_k, texts=texts, dictionary=dictionary, coherence="c_v")
score = cm.get_coherence()
coherence_scores.append(score)
print(f"num_topics={k} coherence(c_v)={score:.4f}")

plt.figure(figsize=(8, 5))
plt.plot(list(topic_range), coherence_scores, marker="o")
plt.xlabel("Number of topics (k)")
plt.ylabel("Coherence score (c_v)")
plt.title("LDA coherence by number of topics")
plt.tight_layout()
plt.show()
```

```
num_topics=3 coherence(c_v)=0.6061
num_topics=4 coherence(c_v)=0.6144
num_topics=5 coherence(c_v)=0.5993
num_topics=6 coherence(c_v)=0.6251
num_topics=7 coherence(c_v)=0.6274
num_topics=8 coherence(c_v)=0.6457
```



```
In [15]: # --- Fit the final LDA model with the best-scoring number of topics ---
NUM_TOPICS = list(topic_range)[int(np.argmax(coherence_scores))]
print(f"Selected NUM_TOPICS = {NUM_TOPICS} (highest coherence)")

# Override NUM_TOPICS manually here if you prefer a more interpretable number, e.g.:
# NUM_TOPICS = 5

lda_model = models.LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=NUM_TOPICS,
    random_state=42,
    passes=50,
    alpha="auto",
    per_word_topics=False,
)

for idx, topic in lda_model.print_topics(num_words=6):
    print(f"Topic {idx}: {topic}")
```

Selected NUM_TOPICS = 8 (highest coherence)

Topic 0: 0.037*"MEDICAL" + 0.035*"WB_1406_DISEASES" + 0.029*"CRISISLEX_C03_WELLBEING_HEALTH" + 0.028*"WB_621_HEALTH_NUTRITION_AND_POPULATION" + 0.024*"GENERAL_HEALTH" + 0.023*"HEALTH_VACCINATION"

Topic 1: 0.069*"WB_635_PUBLIC_HEALTH" + 0.065*"WB_2165_HEALTH_EMERGENCIES" + 0.064*"WB_2166_HEALTH_EMERGENCY_PREPAREDNESS_AND_DISASTER_RESPONSE" + 0.064*"WB_2167_PANDEMICS" + 0.063*"WB_621_HEALTH_NUTRITION_AND_POPULATION" + 0.062*"HEALTH_PANDEMIC"

Topic 2: 0.028*"WB_133_INFORMATION_AND_COMMUNICATION_TECHNOLOGIES" + 0.028*"WB_678_DIGITAL_GOVERNMENT" + 0.027*"WB_694_BROADCAST_AND_MEDIA" + 0.025*"MEDIA_SOCIAL" + 0.023*"WB_652_ICT_APPLICATIONS" + 0.023*"WB_658_ENTERPRISE_APPLICATIONS"

Topic 3: 0.023*"WB_2670_JOBS" + 0.018*"EPU_ECONOMY_HISTORIC" + 0.016*"WB_696_PUBLIC_SECTOR_MANAGEMENT" + 0.014*"WB_1467_EDUCATION_FOR_ALL" + 0.014*"WB_470_EDUCATION" + 0.014*"WB_133_INFORMATION_AND_COMMUNICATION_TECHNOLOGIES"

Topic 4: 0.037*"WB_1104_MACROECONOMIC_VULNERABILITY_AND_DEBT" + 0.030*"TAX_ECON_PRICE" + 0.027*"ECON_INFLATION" + 0.026*"WB_442_INFLATION" + 0.024*"EPU_ECONOMY" + 0.021*"EPU_ECONOMY_HISTORIC"

Topic 5: 0.023*"WB_2432_FRAGILITY_CONFLICT_AND_VIOLENCE" + 0.021*"EPU_CATS_NATIONAL_SECURITY" + 0.019*"ARMEDCONFLICT" + 0.017*"LEADER" + 0.016*"GENERAL_GOVERNMENT" + 0.015*"WB_2470_PEACE_OPERATIONS_AND_CONFLICT_MANAGEMENT"

Topic 6: 0.033*"LEADER" + 0.033*"USPEC_POLITICS_GENERAL1" + 0.022*"ELECTION" + 0.021*"GENERAL_GOVERNMENT" + 0.020*"TAX_FNCACT_PRESIDENT" + 0.018*"EPU_POLICY_POLITICAL"

Topic 7: 0.024*"CRISISLEX_C07_SAFETY" + 0.021*"WB_840_JUSTICE" + 0.021*"WB_696_PUBLIC_SECTOR_MANAGEMENT" + 0.020*"SECURITY_SERVICES" + 0.020*"WB_1014_CRIMINAL_JUSTICE" + 0.019*"CRISISLEX_CRISISLEXREC"

```
In [16]: # --- Assign the dominant topic to each event ---
def get_dominant_topic(bow):
    topic_probs = lda_model.get_document_topics(bow)
    return max(topic_probs, key=lambda x: x[1])[0] if topic_probs else None

df["Topic_ID"] = [get_dominant_topic(bow) for bow in corpus]

# Human-readable Labels: inspect the top words per topic and name them accordingly.
# Edit this dictionary once you've looked at the printed topics above.
topic_labels = {i: f"Topic {i}" for i in range(NUM_TOPICS)}
# Example manual overrides once you've inspected the topics:
# topic_labels = {0: "Political misinformation", 1: "Health misinformation",
#                 2: "Cyber/Information operations", 3: "Economic misinformation", 4: "Media manipulation"}

df["Topic_Label"] = df["Topic_ID"].map(topic_labels)
```

```
df[["Record_ID", "Themes", "Topic_ID", "Topic_Label"]].head(10)

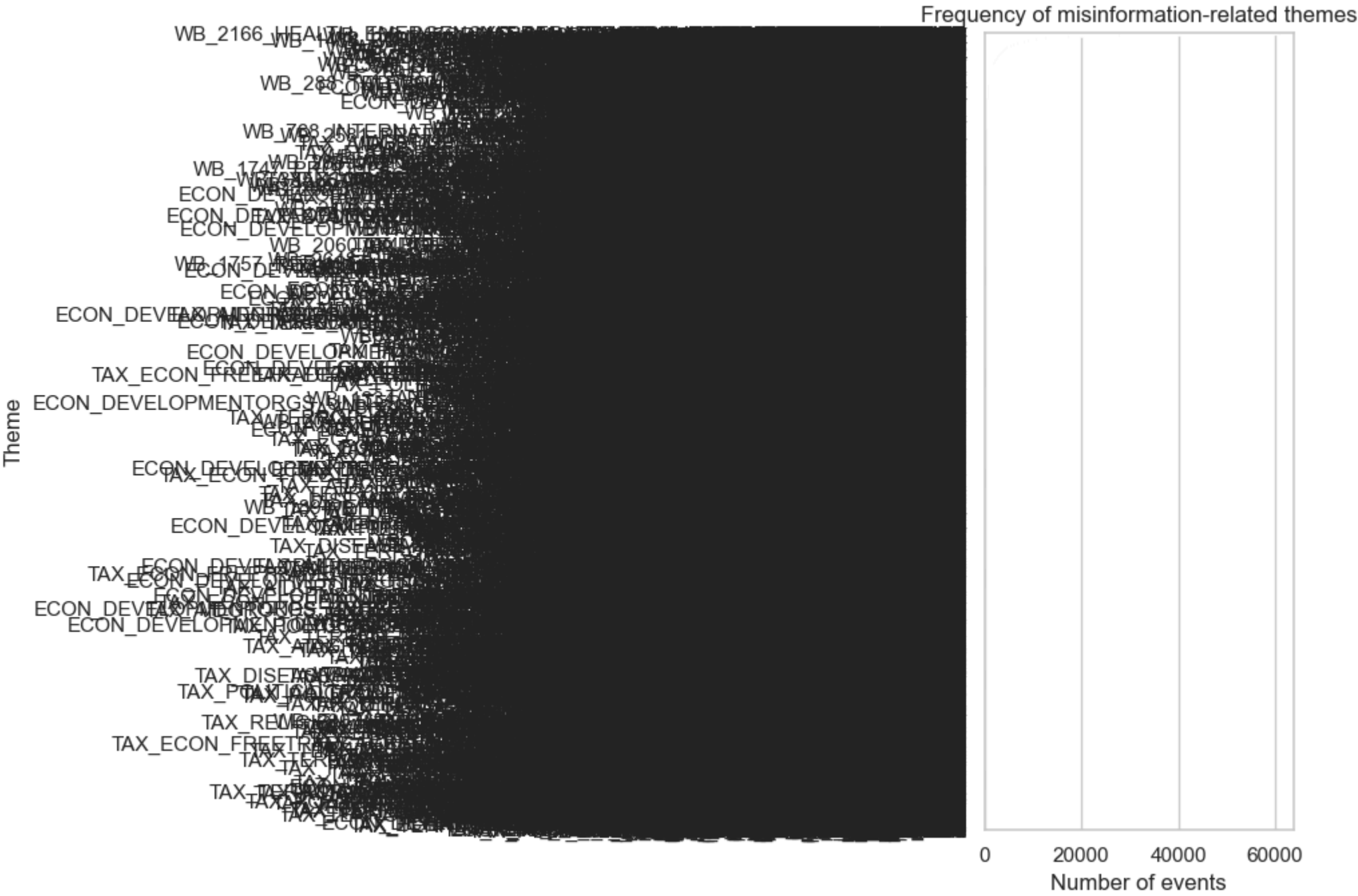
Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style>
<thead> <th></th> <th>Record_ID</th> <th>Themes</th> <th>Topic_ID</th> <th>Topic_Label</th> </thead>
<tbody> <tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>
<tr><td></td> <td></td> <td></td> <td></td> <td></td> </tr>

<h2 id="5.-Exploratory-analysis-of-themes-and-tone">5. Exploratory analysis of themes and tone</h2>
```

```
In [17]: # --- Theme frequency across all events ---
theme_counts = df["Theme_Tokens"].explode().value_counts()

plt.figure(figsize=(10, 7))
sns.barplot(x=theme_counts.values, y=theme_counts.index, color="steelblue")
plt.xlabel("Number of events")
plt.ylabel("Theme")
plt.title("Frequency of misinformation-related themes")
plt.tight_layout()
plt.show()

theme_counts
```



```
Out[17]: Theme_Tokens
GENERAL_HEALTH 60815
WB_621_HEALTH_NUTRITION_AND_POPULATION 60755
UNGP_HEALTHCARE 56663
WB_635_PUBLIC_HEALTH 53040
WB_2165_HEALTH_EMERGENCIES 50795
...
TAX_DISEASE_SCHWANNOMA 1
TAX_WORLD_BIRDS_KILLDEER 1
TAX_POLITICAL_PARTY_POPULAR_FRONT_PARTY 1
TAX_DISEASE_WILMS_TUMOR 1
TAX_FNCACT_SEAMANS 1
Name: count, Length: 10733, dtype: int64
```

```
In [18]: # --- Dominant narratives per country (top theme per country) ---
theme_by_country = (
    df.explode("Theme_Tokens")
      .groupby(["Country", "Theme_Tokens"])
      .size()
      .reset_index(name="Count")
)

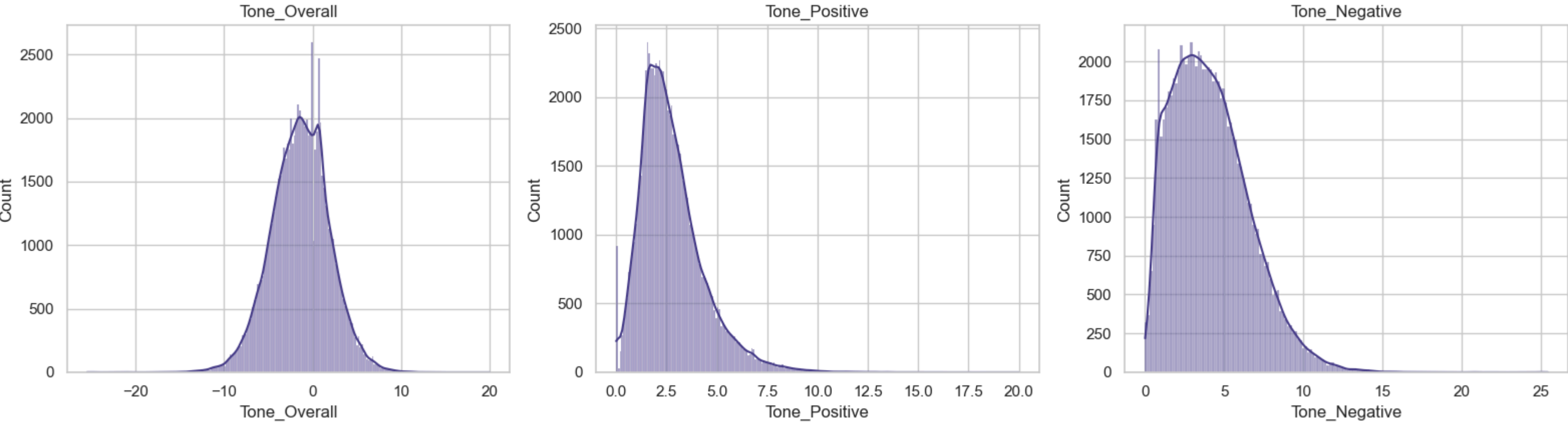
top_theme_per_country = (
    theme_by_country
      .sort_values("Count", ascending=False)
      .groupby("Country")
      .first()
      .sort_values("Count", ascending=False)
)
top_theme_per_country
```

Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Theme_Tokens</th> <th>Count</th> <th>Country</th> <th></th> <th></th> </thead> <tbody> <th>USA</th> <th>United Kingdom</th> <th>Russia</th> <th>India</th> <th>American</th> <th>...</th> <th>Bahrainis</th> <th>Arab Republic Of Egypt</th> <th>Uzbeks</th> <th>Union Of South Africa</th> <th>Uzbekistani</th> </tbody> <tbody></tbody>

WB_621_HEALTH_NUTRITION_AND_POPULATION	16099
GENERAL_HEALTH	8148
TAX_WORLDLANGUAGES_RUSSIA	5477
GENERAL_HEALTH	3962
UNGP_FORESTS_RIVERS_OCEANS	2008
...	...
WB_635_PUBLIC_HEALTH	1
WB_895_MINING_SYSTEMS	1
EPU_POLICY_AUTHORITIES	1
TAX_FNCACT_CHAIRPERSON	1
TAX_FNCACT_CHEF	1

<p>563 rows × 2 columns</p>

```
In [19]: # --- Tone distributions ---
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for ax, col in zip(axes, ["Tone_Overall", "Tone_Positive", "Tone_Negative"]):
    sns.histplot(df[col], kde=True, ax=ax, color="darkslateblue")
    ax.set_title(col)
plt.tight_layout()
plt.show()
```



```
In [20]: # --- Tone by geographic resolution and by country ---
# Location_Type: GDELT's own resolution code (1=Country, 2=US State, 3=US City,
# 4=World City, 5=World State) for the primary location matched in each article.
location_type_labels = {1: "Country", 2: "US State", 3: "US City", 4: "World City", 5: "World State"}
df["Location_Type_Label"] = df["Location_Type"].map(location_type_labels).fillna("Other")

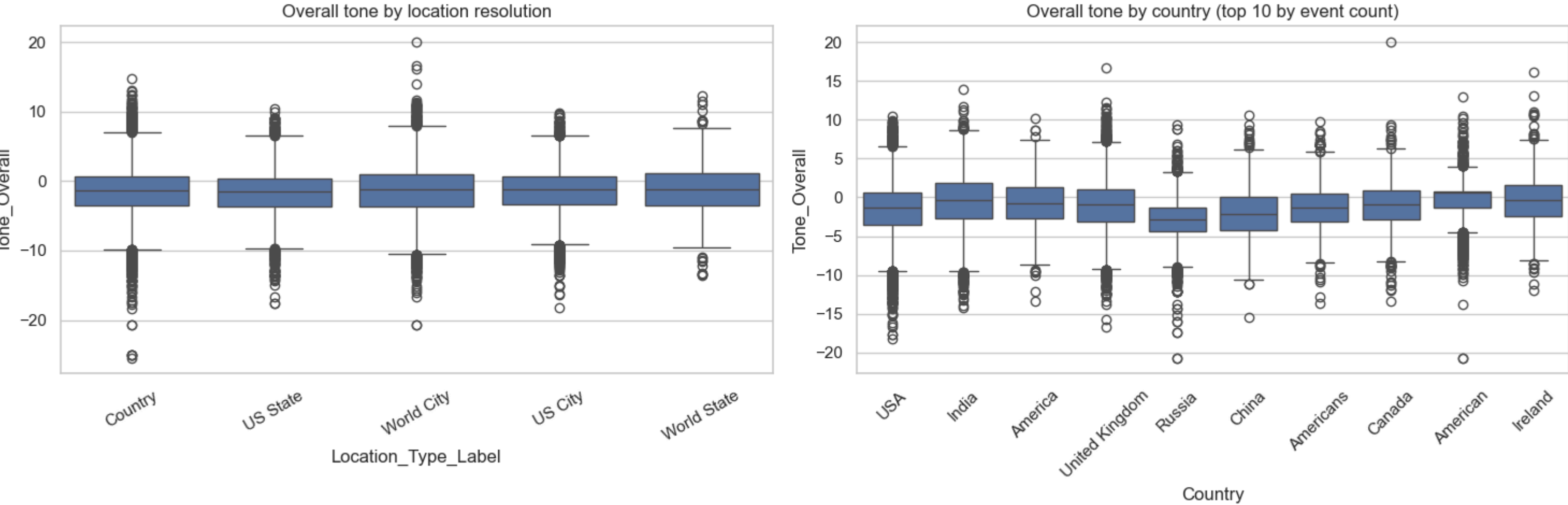
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

sns.boxplot(data=df, x="Location_Type_Label", y="Tone_Overall", ax=axes[0])
axes[0].set_title("Overall tone by location resolution")
axes[0].tick_params(axis="x", rotation=30)

top_countries = df["Country"].value_counts().head(10).index
sns.boxplot(data=df[df["Country"].isin(top_countries)], x="Country", y="Tone_Overall", ax=axes[1])
axes[1].set_title("Overall tone by country (top 10 by event count)")
axes[1].tick_params(axis="x", rotation=45)
```



```
plt.tight_layout()
plt.show()
```



```
In [21]: # --- Topic vs. tone: mean tone scores per LDA topic ---
topic_tone = df.groupby("Topic_Label")[["Tone_Overall", "Tone_Positive", "Tone_Negative", "Tone_Polarity"]].mean().round(2)
topic_tone
```

```
Out[21]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Tone_Overall</th> <th>Tone_Positive</th> <th>Tone_Negative</th> <th>Tone_Polarity</th> <th>Topic_Label</th> <th></th> <th></th>
<th></th> <th></th> </thead> <tbody> <th>Topic 0</th> <th>Topic 1</th> <th>Topic 2</th> <th>Topic 3</th> <th>Topic 4</th> <th>Topic 5</th> <th>Topic 6</th> <th>Topic 7</th> </tbody> <tbody></tbody>
```

-1.84	2.89	4.73	7.61
-0.40	2.92	3.32	6.24
-0.03	3.28	3.32	6.60
0.27	3.26	2.98	6.24
-0.91	2.08	2.99	5.07
-2.69	2.37	5.05	7.42
-1.87	2.84	4.70	7.54
-4.41	2.17	6.58	8.74

<h2 id="6.-Geographic-clustering-and-hotspots-(spatial-analysis)">6. Geographic clustering and hotspots (spatial analysis)</h2>

```
In [22]: # --- Convert to a GeoDataFrame ---
geometry = [Point(xy) for xy in zip(df["Longitude"], df["Latitude"])]
gdf = gpd.GeoDataFrame(df.copy(), geometry=geometry, crs="EPSG:4326")
gdf.head()
```

```
Out[22]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Record_ID</th> <th>Date</th> <th>Year</th> <th>Month</th> <th>Year_Month</th> <th>Source</th> <th>Source_URL</th>
<th>Country</th> <th>Location_Type</th> <th>Latitude</th> <th>...</th> <th>Tone_Positive</th> <th>Tone_Negative</th> <th>Tone_Polarity</th> <th>Tone_Activity</th> <th>Tone_SelfGroup</th> <th>Theme_Tokens</th> <th>Topic_ID</th> <th>Topic_Label</th> <th>Location_Type_Label</th> <th>geometry</th>
</thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>
```

20220101121500-2	2022-01-01 12:15:00	2022	1	2022-01	smdailyjournal.com	https://www.smdailyjournal.com/news/world/pope...	Vatican City	1	41.9000	...	3.676471	3.921569	7.598039	21.323529	0.490196	[TAX_FNCACT_CARDINAL, WB_1467_EDUCATION_FOR_AL...	1	Topic 1	Country	POINT (12.45 41.9)
20220101121500-4	2022-01-01 12:15:00	2022	1	2022-01	townhall.com	https://townhall.com/tipsheet/mattvespa/2022/0...	USA	2	32.9866	...	1.246106	4.049844	5.295950	25.233645	2.024922	[HEALTH_VACCINATION, UNGP_HEALTHCARE, TAX_FNCA...	6	Topic 6	US State	POINT (-83.6487 32.9866)
20220101121500-5	2022-01-01 12:15:00	2022	1	2022-01	gov.ms	https://www.gov.ms/2022/01/01/new-years-messag...	Montserrat	1	16.7500	...	4.691618	2.108593	6.800211	22.298366	2.899315	[SHORTAGE, UNGP_FORESTS_RIVERS_OCEANS, WB_528_...	3	Topic 3	Country	POINT (-62.2 16.75)
20220101121500-6	2022-01-01 12:15:00	2022	1	2022-01	majorcadailybulletin.com	https://www.majorcadailybulletin.com/news/loca...	Spain	4	38.3462	...	4.716981	0.943396	5.660377	17.358491	0.000000	[GENERAL_GOVERNMENT, EPU_POLICY_GOVERNMENT, GE...	1	Topic 1	World City	POINT (-0.48772 38.3462)
20220101121500-8	2022-01-01 12:15:00	2022	1	2022-01	houstonchronicle.com	https://www.houstonchronicle.com/news/article/...	USA	3	44.0805	...	1.595298	0.503778	2.099076	22.670025	2.099076	[RURAL, TAX_FNCACT_ASSISTANT, TAX_FNCACT_GUIDE...	1	Topic 1	US City	POINT (-103.231 44.0805)

<p>5 rows × 24 columns</p>

```
In [23]: # --- Plot events over a world basemap ---
# Requires the 'naturalearth_lowres' dataset shipped with older geopandas, or download one yourself:
# world = gpd.read_file(gpd.datasets.get_path("naturalearth_lowres"))
# If gpd.datasets is unavailable in your geopandas version, download a shapefile from
# https://www.naturalearthdata.com and load it with gpd.read_file("path/to/ne_110m_admin_0_countries.shp")

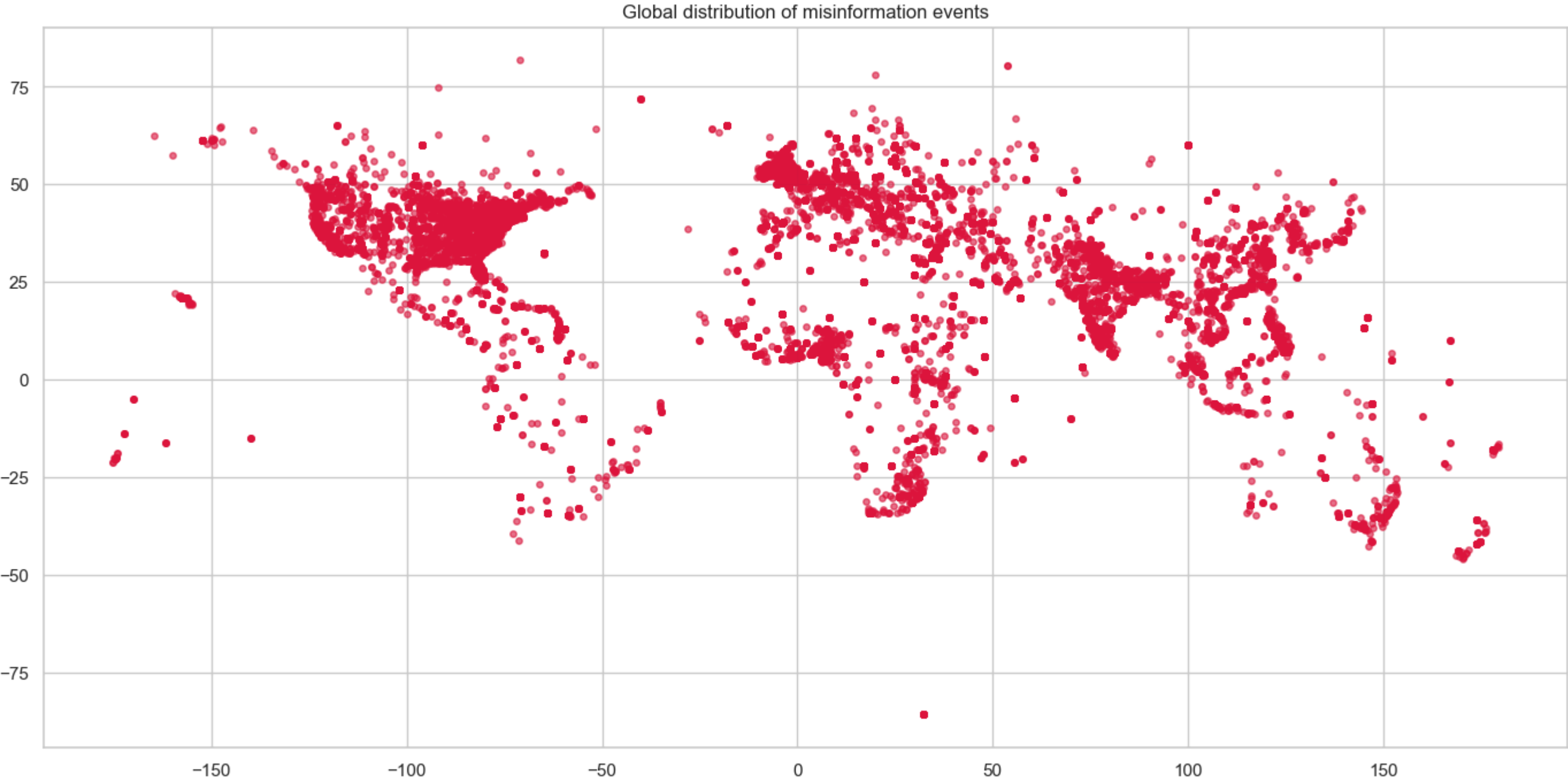
try:
    world = gpd.read_file(gpd.datasets.get_path("naturalearth_lowres"))
except Exception as e:
    print("Could not load built-in basemap:", e)
    print("Download a countries shapefile from naturalearthdata.com and load it with gpd.read_file(...)")
    world = None

fig, ax = plt.subplots(figsize=(14, 8))
if world is not None:
```



```
world.plot(ax=ax, color="whitesmoke", edgecolor="lightgray")
gdf.plot(ax=ax, markersize=15, color="crimson", alpha=0.6)
ax.set_title("Global distribution of misinformation events")
plt.tight_layout()
plt.show()
```

Could not load built-in basemap: The geopandas.dataset has been deprecated and was removed in GeoPandas 1.0. You can get the original 'naturalearth_lowres' data from <https://www.naturalearthdata.com/downloads/110m-cultural-vectors/>.
Download a countries shapefile from [naturalearthdata.com](https://www.naturalearthdata.com) and load it with `gpd.read_file(...)`

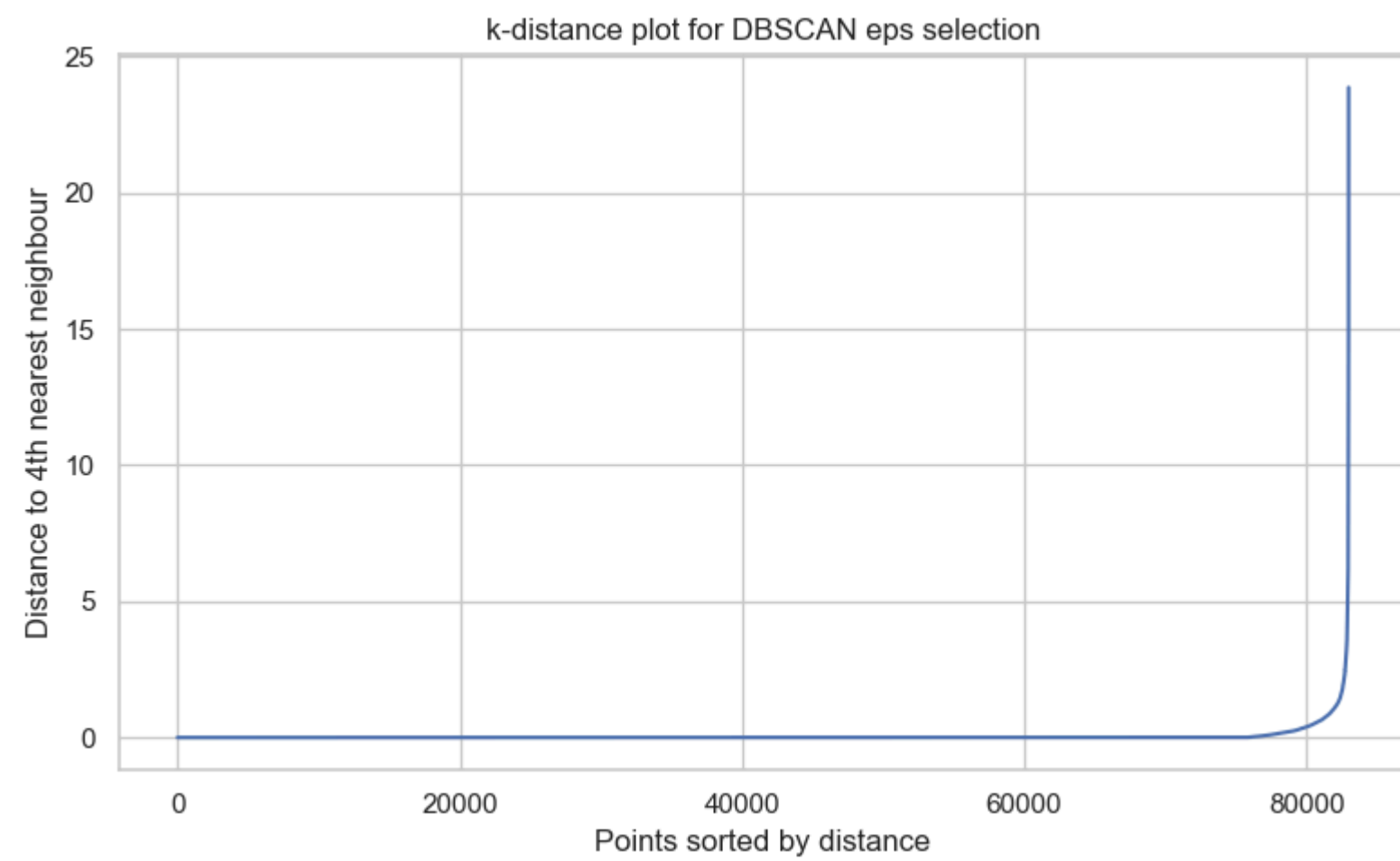


In [24]:

```
# --- DBSCAN clustering ---
coords = gdf[["Longitude", "Latitude"]].to_numpy()

# k-distance plot to help choose eps (Look for the "elbow")
MIN_SAMPLES = 4
neighbors = NearestNeighbors(n_neighbors=MIN_SAMPLES).fit(coords)
distances, _ = neighbors.kneighbors(coords)
k_distances = np.sort(distances[:, -1])

plt.figure(figsize=(8, 5))
plt.plot(k_distances)
plt.xlabel("Points sorted by distance")
plt.ylabel(f"Distance to {MIN_SAMPLES}th nearest neighbour")
plt.title("k-distance plot for DBSCAN eps selection")
plt.tight_layout()
plt.show()
```



In [25]:

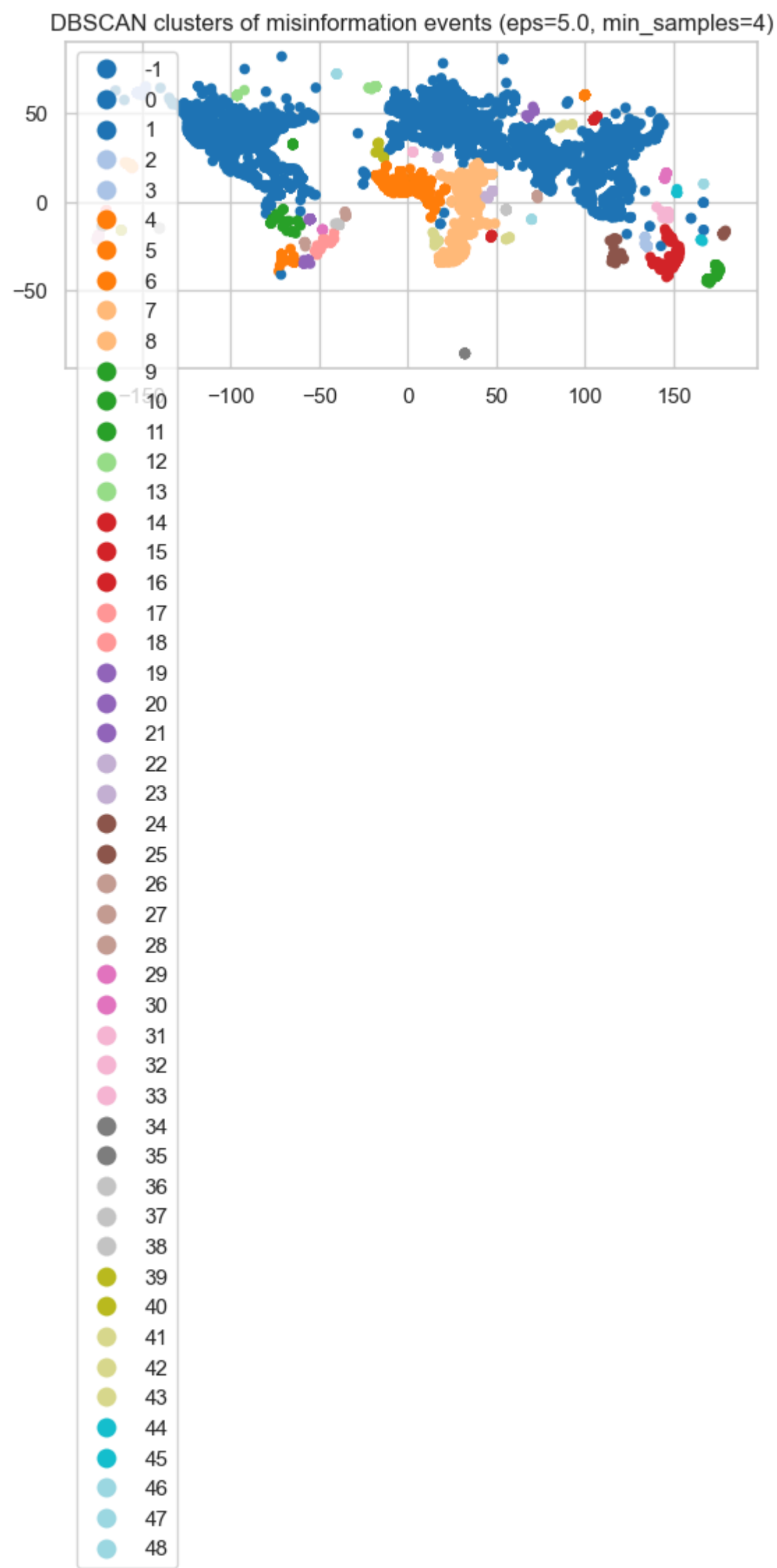
```
# Set eps based on the elbow in the plot above (degrees of Lat/Lon, since coordinates aren't projected)
EPS = 5.0          # <-- adjust after inspecting the k-distance plot
MIN_SAMPLES = 4

db = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES).fit(coords)
gdf["Cluster_ID"] = db.labels_

n_clusters = len(set(db.labels_)) - (1 if -1 in db.labels_ else 0)
n_noise = list(db.labels_).count(-1)
print(f"DBSCAN found {n_clusters} clusters and {n_noise} noise points (label -1)")

fig, ax = plt.subplots(figsize=(14, 8))
if world is not None:
    world.plot(ax=ax, color="whitesmoke", edgecolor="lightgray")
gdf.plot(ax=ax, column="Cluster_ID", categorical=True, markersize=20, legend=True, cmap="tab20")
ax.set_title(f"DBSCAN clusters of misinformation events (eps={EPS}, min_samples={MIN_SAMPLES})")
plt.tight_layout()
plt.show()
```

DBSCAN found 49 clusters and 45 noise points (label -1)



In [26]:

```
# --- Moran's I: spatial autocorrelation of event counts at the country level ---
# Aggregate to country level
country_counts = df.groupby("Country").agg(
    Misinfo_Event_Count=("Record_ID", "count"),
    Mean_Tone=("Tone_Overall", "mean"),
    Latitude=("Latitude", "mean"),
    Longitude=("Longitude", "mean"),
).reset_index()

country_gdf = gpd.GeoDataFrame(
    country_counts,
    geometry=[Point(xy) for xy in zip(country_counts["Longitude"], country_counts["Latitude"])],
    crs="EPSG:4326",
)

# Distance/contiguity-based spatial weights (K-nearest-neighbour weights on country centroids)
w = KNN.from_dataframe(country_gdf, k=4)
w.transform = "r"

moran_counts = Moran(country_gdf["Misinfo_Event_Count"].values, w)
moran_tone = Moran(country_gdf["Mean_Tone"].values, w)

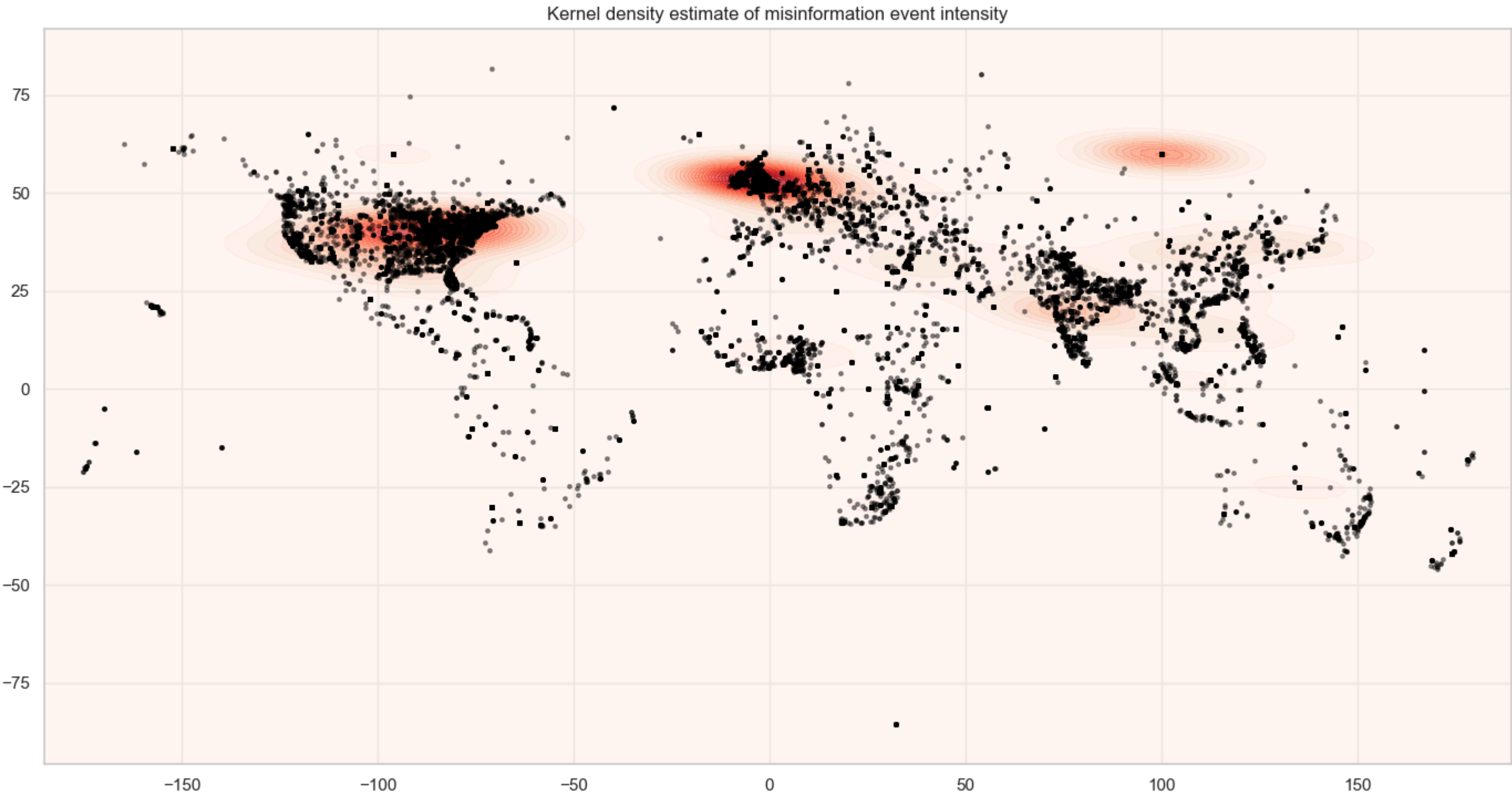
print(f"Moran's I (event counts): I={moran_counts.I:.4f}, p-value={moran_counts.p_sim:.4f}")
print(f"Moran's I (mean tone): I={moran_tone.I:.4f}, p-value={moran_tone.p_sim:.4f}")
```


Moran's I (event counts): I=0.1245, p-value=0.0070
Moran's I (mean tone): I=0.1157, p-value=0.0010

```
In [27]: # --- Kernel Density Estimation of event intensity ---
xy = np.vstack([gdf["Longitude"], gdf["Latitude"]])
kde = gaussian_kde(xy)

xmin, xmax = gdf["Longitude"].min() - 10, gdf["Longitude"].max() + 10
ymin, ymax = gdf["Latitude"].min() - 10, gdf["Latitude"].max() + 10
xx, yy = np.mgrid[xmin:xmax:200j, ymin:ymax:200j]
positions = np.vstack([xx.ravel(), yy.ravel()])
density = np.reshape(kde(positions).T, xx.shape)

fig, ax = plt.subplots(figsize=(14, 8))
if world is not None:
    world.plot(ax=ax, color="none", edgecolor="lightgray")
ax.contourf(xx, yy, density, levels=25, cmap="Reds", alpha=0.7)
gdf.plot(ax=ax, markersize=6, color="black", alpha=0.4)
ax.set_title("Kernel density estimate of misinformation event intensity")
plt.tight_layout()
plt.show()
```



<h2 id="7.-Construct-contextual-variables">7. Construct contextual variables</h2>

```
In [28]: # --- Infer political / economic / health context from theme tags ---
# NOTE: every code below is a REAL, VERIFIED GKG theme code -- confirmed either against
# GDELT's official theme master list (http://data.gdeltproject.org/documentation/
# GKG-MASTER-THEMELIST.TXT) or by directly observing it in this project's own
# raw_gkg_extraction.csv. Do not add a code here unless you've verified it the same way --
# this is exactly the step that went wrong with the original synthetic dataset.

POLITICAL_THEMES = {"GENERAL_GOVERNMENT", "EPU_POLICY", "EPU_POLICY_GOVERNMENT",
                    "TAX_FNCACT_POLITICIAN", "TAX_FNCACT_PRESIDENT"}
ECONOMIC_THEMES  = {"ECON_INFLATION", "EPU_ECONOMY", "WB_442_INFLATION"}
HEALTH_THEMES    = {"GENERAL_HEALTH", "HEALTH_PANDEMIC", "HEALTH_VACCINATION"}

def flag_context(tokens, keyword_set):
    return int(len(set(tokens) & keyword_set) > 0)

df["Political_Context"] = df["Theme_Tokens"].apply(lambda t: flag_context(t, POLITICAL_THEMES))
df["Economic_Context"] = df["Theme_Tokens"].apply(lambda t: flag_context(t, ECONOMIC_THEMES))
df["Health_Context"]   = df["Theme_Tokens"].apply(lambda t: flag_context(t, HEALTH_THEMES))

df[["Political_Context", "Economic_Context", "Health_Context"]].mean().rename("Share of events")
```

Out[28]: Political_Context 0.514776
Economic_Context 0.292224
Health_Context 0.740593
Name: Share of events, dtype: float64


```
In [29]: # --- Aggregate to Country x Year_Month Level ---
```

```
country_month = df.groupby(["Country", "Year_Month"]).agg(
    Misinfo_Event_Count=("Record_ID", "count"),
    Political_Prop=("Political_Context", "mean"),
    Economic_Prop=("Economic_Context", "mean"),
    Health_Prop=("Health_Context", "mean"),
    Tone_Overall_Mean=("Tone_Overall", "mean"),
    Source_Diversity=("Source", "nunique"),
).reset_index()

print(country_month.shape)
country_month.head(10)
```

 $(5261, 8)$

```
Out[29]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }.dataframe tbody tr th { vertical-align: top; }.dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Country</th> <th>Year_Month</th> <th>Misinfo_Event_Count</th> <th>Political_Prop</th> <th>Economic_Prop</th> <th>Health_Prop</th> <th>Tone_Overall_Mean</th> <th>Source_Diversity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody> </tbody>
```

Afghan	2022-04	1	0.0	0.000000	1.000000	0.566251	1
Afghan	2023-03	1	1.0	1.000000	1.000000	-2.959184	1
Afghan	2023-04	3	1.0	0.000000	1.000000	-2.875399	3
Afghan	2023-05	6	0.5	0.000000	0.666667	-1.797944	6
Afghan	2023-06	3	1.0	0.666667	0.666667	-3.489921	3
Afghan	2023-07	4	0.5	0.500000	1.000000	-0.547628	3
Afghan	2023-08	2	0.5	0.500000	0.000000	-2.776013	2
Afghan	2023-09	1	1.0	0.000000	0.000000	-3.703704	1
Afghan	2023-12	1	1.0	1.000000	1.000000	-3.581662	1
Afghani	2022-01	1	1.0	0.000000	1.000000	-4.702970	1

8. Contextual drivers via count-based regression

```
In [30]: # --- Prepare the modelling dataset (Country-Month rows) ---
```

```
model_df = country_month.copy()
model_df = model_df.dropna(subset=["Misinfo_Event_Count", "Political_Prop", "Economic_Prop", "Health_Prop"])
model_df.describe()
```

```
Out[30]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Misinfo_Event_Count</th> <th>Political_Prop</th> <th>Economic_Prop</th> <th>Health_Prop</th> <th>Tone_Overall_Mean</th> <th>Source_Diversity</th> </thead> <tbody> <tr> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody> </tbody>
```

5261.000000	5261.000000	5261.000000	5261.000000	5261.000000	5261.000000
15.771526	0.560893	0.302292	0.706888	-1.295439	10.954761
73.092110	0.378815	0.348065	0.354870	2.809745	41.881462
1.000000	0.000000	0.000000	0.000000	-16.666667	1.000000
1.000000	0.250000	0.000000	0.500000	-2.890292	1.000000
2.000000	0.562500	0.200000	0.852459	-1.218170	2.000000
7.000000	1.000000	0.500000	1.000000	0.351288	6.000000
1236.000000	1.000000	1.000000	1.000000	11.406844	667.000000

```
In [31]: # --- Poisson regression ---
         poisson_formula = "Misinfo_Event_Count ~ Political_Prop + Economic_Prop + Health_Prop + Tone_Overall_Mean"
```

```
poisson_model = smf.glm(
    formula=poisson_formula,
    data=model_df,
    family=sm.families.Poisson(),
).fit()
```

```
print(poisson_model.summary())
```

Generalized Linear Model Regression Results						
=====						
Dep. Variable:	Misinfo_Event_Count	No. Observations:	5261			
Model:	GLM	Df Residuals:	5256			
Model Family:	Poisson	Df Model:	4			
Link Function:	Log	Scale:	1.0000			
Method:	IRLS	Log-Likelihood:	-1.7058e+05			
Date:	Mon, 24 Aug 2026	Deviance:	3.2458e+05			
Time:	18:17:07	Pearson chi2:	1.72e+06			
No. Iterations:	7	Pseudo R-squ. (CS):	0.3688			
Covariance Type:	nonrobust					
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	2.6809	0.010	268.300	0.000	2.661	2.700
Political_Prop	-0.3323	0.009	-35.329	0.000	-0.351	-0.314
Economic_Prop	-0.0510	0.011	-4.830	0.000	-0.072	-0.030
Health_Prop	0.3283	0.011	30.868	0.000	0.307	0.349
Tone_Overall_Mean	-0.0250	0.001	-19.710	0.000	-0.027	-0.023
=====						

```
In [32]: # --- Check for overdispersion (variance vs mean of the dependent variable) ---
mean_count = model_df["Misinfo_Event_Count"].mean()
var_count = model_df["Misinfo_Event_Count"].var()
dispersion_ratio = var_count / mean_count

print(f"Mean(Misinfo_Event_Count) = {mean_count:.3f}")
print(f"Var(Misinfo_Event_Count) = {var_count:.3f}")
print(f"Dispersion ratio (var/mean) = {dispersion_ratio:.3f}")
print("> Overdispersion suspected (fit Negative Binomial)" if dispersion_ratio > 1.5 else "> Poisson assumption looks reasonable")

# Pearson chi2 / degrees of freedom is another standard overdispersion diagnostic
pearson_chi2 = poisson_model.pearson_chi2
df_resid = poisson_model.df_resid
print(f"Pearson chi2 / df = {pearson_chi2 / df_resid:.3f} (values well above 1 indicate overdispersion)")

Mean(Misinfo_Event_Count) = 15.772
Var(Misinfo_Event_Count) = 5342.457
Dispersion ratio (var/mean) = 338.741
> Overdispersion suspected (fit Negative Binomial)
Pearson chi2 / df = 326.536 (values well above 1 indicate overdispersion)
```

```
In [33]: # --- Negative Binomial regression (fit if overdispersion is present) ---
# NB: smf.glm(family=sm.families.NegativeBinomial()) fixes alpha=1.0 by default, which
# is rarely the true dispersion in your data (hence the ValueWarning). Instead, we use
# smf.negativebinomial(), which estimates alpha from the data via maximum likelihood.
nb_model = smf.negativebinomial(
    formula=poisson_formula,
    data=model_df,
).fit(dis=False)

print(nb_model.summary())
print(f"\nEstimated dispersion parameter alpha = {nb_model.params['alpha']:.4f}")
```

NegativeBinomial Regression Results						
=====						
Dep. Variable:	Misinfo_Event_Count	No. Observations:	5261			
Model:	NegativeBinomial	Df Residuals:	5256			
Method:	MLE	Df Model:	4			
Date:	Mon, 24 Aug 2026	Pseudo R-squ.:	0.006469			
Time:	18:17:12	Log-Likelihood:	-18142.			
converged:	True	LL-Null:	-18260.			
Covariance Type:	nonrobust	LLR p-value:	5.935e-50			
=====						
	coef	std err	z	P> z	[0.025	0.975]

Intercept	2.5984	0.093	27.808	0.000	2.415	2.781
Political_Prop	-1.3230	0.111	-11.928	0.000	-1.540	-1.106
Economic_Prop	0.0245	0.122	0.201	0.841	-0.214	0.263
Health_Prop	0.9722	0.102	9.558	0.000	0.773	1.172
Tone_Overall_Mean	-0.1174	0.015	-7.936	0.000	-0.146	-0.088
alpha	2.3453	0.039	59.700	0.000	2.268	2.422
=====						

Estimated dispersion parameter alpha = 2.3453

```
In [34]: # --- Interpret coefficients as Incidence Rate Ratios (IRRs) ---
def irr_table(fitted_model):
    params = fitted_model.params.drop(labels=["alpha"], errors="ignore") # alpha is a dispersion param, not a rate ratio
    conf = fitted_model.conf_int().drop(index=["alpha"], errors="ignore")
    conf.columns = ["ci_lower", "ci_upper"]
    out = pd.DataFrame({
        "coef": params,
        "IRR": np.exp(params),
        "IRR_ci_lower": np.exp(conf["ci_lower"]),
        "IRR_ci_upper": np.exp(conf["ci_upper"]),
        "p_value": fitted_model.pvalues.drop(labels=["alpha"], errors="ignore"),
    })
    return out.round(3)

print("Poisson model - Incidence Rate Ratios")
display(irr_table(poisson_model))

print("\nNegative Binomial model - Incidence Rate Ratios")
display(irr_table(nb_model))
```

Poisson model - Incidence Rate Ratios

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }.dataframe tbody tr th { vertical-align: top; }.dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>coef</th> <th>IRR</th> <th>IRR_ci_lower</th> <th>IRR_ci_upper</th> <th>p_value</th> </thead> <tbody> <th>Intercept</th>
<th>Political_Prop</th> <th>Economic_Prop</th> <th>Health_Prop</th> <th>Tone_Overall_Mean</th> </tbody> <tbody></tbody>

2.681	14.598	14.315	14.886	0.0
-------	--------	--------	--------	-----

-0.332	0.717	0.704	0.731	0.0
--------	-------	-------	-------	-----

-0.051	0.950	0.931	0.970	0.0
--------	-------	-------	-------	-----

0.328	1.389	1.360	1.418	0.0
-------	-------	-------	-------	-----

-0.025	0.975	0.973	0.978	0.0
--------	-------	-------	-------	-----

Negative Binomial model - Incidence Rate Ratios

<style scoped> .dataframe tbody tr th { vertical-align: middle; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>coef</th> <th>IRR</th> <th>IRR_ci_lower</th> <th>IRR_ci_upper</th> <th>p_value</th> </thead> <tbody> <th>Intercept</th>
<th>Political_Prop</th> <th>Economic_Prop</th> <th>Health_Prop</th> <th>Tone_Overall_Mean</th> </tbody> <tbody> </tbody>

2.598	13.442	11.192	16.143	0.000
-1.323	0.266	0.214	0.331	0.000
0.024	1.025	0.807	1.301	0.841
0.972	2.644	2.166	3.227	0.000
-0.117	0.889	0.864	0.915	0.000

<h2 id="9.-Final-analysis-and-visualisation-outputs">9. Final analysis and visualisation outputs</h2>

```
In [35]: # --- Key findings summary tables ---
print("=== Dominant misinformation themes ===")
print(theme_counts.head(5), "\n")

print("=== Mean tone by topic ===")
print(topic_tone, "\n")

print("=== Countries with highest event counts ===")
print(country_counts.sort_values("Misinfo_Event_Count", ascending=False).head(5), "\n")

print("=== Moran's I results ===")
print(f"Event counts: I={moran_counts.I:.4f} (p={moran_counts.p_sim:.4f})")
print(f"Mean tone:    I={moran_tone.I:.4f} (p={moran_tone.p_sim:.4f})\n")

print("=== Negative Binomial IRRs (contextual drivers) ===")
print(irr_table(nb_model))

=== Dominant misinformation themes ===
Theme_Tokens
GENERAL_HEALTH          60815
WB_621_HEALTH_NUTRITION_AND_POPULATION  60755
UNGP_HEALTHCARE          56663
WB_635_PUBLIC_HEALTH     53040
WB_2165_HEALTH_EMERGENCIES  50795
Name: count, dtype: int64

=== Mean tone by topic ===
      Topic_Label  Tone_Overall  Tone_Positive  Tone_Negative  Tone_Polarity
Topic 0          -1.84         2.89         4.73         7.61
Topic 1          -0.40         2.92         3.32         6.24
Topic 2          -0.03         3.28         3.32         6.60
Topic 3           0.27         3.26         2.98         6.24
Topic 4          -0.91         2.08         2.99         5.07
Topic 5          -2.69         2.37         5.05         7.42
Topic 6          -1.87         2.84         4.70         7.54
Topic 7          -4.41         2.17         6.58         8.74

=== Countries with highest event counts ===
      Country  Misinfo_Event_Count  Mean_Tone  Latitude  Longitude
526      USA          20659  -1.500929  38.113062  -90.596193
535  United Kingdom      10637  -0.966016  53.413922  -2.808155
420      Russia          6005  -2.853336  59.727255  96.211262
214      India          5712  -0.548717  20.933705  77.318904
10    American          2690  -0.255848  39.828175  -98.579500

=== Moran's I results ===
Event counts: I=0.1245 (p=0.0070)
Mean tone:    I=0.1157 (p=0.0010)

=== Negative Binomial IRRs (contextual drivers) ===
      coef    IRR  IRR_ci_lower  IRR_ci_upper  p_value
Intercept    2.598  13.442         11.192         16.143    0.000
Political_Prop -1.323    0.266          0.214          0.331    0.000
Economic_Prop  0.024    1.025          0.807          1.301    0.841
Health_Prop    0.972    2.644          2.166          3.227    0.000
Tone_Overall_Mean -0.117    0.889          0.864          0.915    0.000
```

```
In [36]: # --- Effect plot: IRRs with confidence intervals ---
irr_df = irr_table(nb_model).drop(index="Intercept", errors="ignore")

plt.figure(figsize=(8, 5))
plt.errorbar(
    irr_df["IRR"], irr_df.index,
    xerr=[irr_df["IRR"] - irr_df["IRR_ci_lower"], irr_df["IRR_ci_upper"] - irr_df["IRR"]],
    fmt="o", capsize=4, color="firebrick",
)
plt.axvline(1, linestyle="--", color="gray")
plt.xlabel("Incidence Rate Ratio (IRR)")
plt.title("Contextual drivers of misinformation event counts (Negative Binomial)")
plt.tight_layout()
plt.show()
```




```
In [37]: # --- Save key outputs for use in your report/thesis chapter ---
import os
OUT_DIR = "outputs"
os.makedirs(OUT_DIR, exist_ok=True)

df.to_csv(f"{OUT_DIR}/cleaned_events_with_topics.csv", index=False)
country_month.to_csv(f"{OUT_DIR}/country_month_panel.csv", index=False)
theme_counts.to_csv(f"{OUT_DIR}/theme_frequencies.csv")
topic_tone.to_csv(f"{OUT_DIR}/topic_vs_tone.csv")
irr_table(nb_model).to_csv(f"{OUT_DIR}/negative_binomial_irr.csv")

print(f"Saved cleaned data, panel dataset, and result tables to ./{OUT_DIR}/")
```

Saved cleaned data, panel dataset, and result tables to ./outputs/

In []: